

BAB II

TINJAUAN PUSTAKA

2.1 Konsep atau Teori yang Relevan

2.1.1 Kurs atau Nilai Tukar

Kurs atau yang lebih umum disebut sebagai nilai tukar mata uang merupakan perbedaan nilai yang dimiliki mata uang suatu negara terhadap negara lain. Krugman dan Obsfeld (2004) dalam penelitian menjelaskan bahwa Kurs atau nilai tukar adalah harga yang dimiliki mata uang dari suatu negara yang diukur berdasarkan harga mata uang negara lain. Kurs mempunyai peranan penting dalam keputusan transaksi pembelian dikarenakan kurs memungkinkan para penggunanya mengonversi harga-harga dari berbagai negara ke dalam satu harga atau nilai yang sama. Menurut Mishkin (2009) menjelaskan secara singkat bahwa kurs adalah harga atau nilai suatu mata uang dalam mata uang lainnya.

Perbedaan nilai tukar bisa terjadi umumnya di pasar uang karena melalui mekanisme keseimbangan permintaan dan penawaran mata uang asing yang dihitung berdasarkan mata uang suatu negara (Effendie, 2017). Nilai tukar menurut (Hadi, 2015), bisa disebut sebagai kurs valuta dalam berbagai transaksi yang dilakukan oleh negara lain yang di dalamnya terdapat selisih nilai di antara kedua negara tersebut, hal inilah yang menciptakan nilai tukar atau kurs (Ramadhan & Simamora, 2022).

2.1.2 Data Preprocessing

Pra-pemrosesan data atau *Data Preprocessing* merupakan salah satu tahap kunci yang dilakukan dalam proses analisis data. Pada tahap ini, data mentah diubah ke dalam format yang dapat dipahami dan dievaluasi oleh komputer serta algoritma machine learning (Ramírez-Gallego, dkk., 2017). Pra-pemrosesan data mempermudah dalam mendeteksi dan menangani kesalahan, outlier, serta informasi yang hilang. Proses ini menghasilkan data yang lebih bersih dan akurat, yang pada akhirnya memungkinkan analisis dan pemodelan yang lebih tepat (Brijith, 2023). Berikut beberapa tahapan pra-pemrosesan data.

2.1.2.1 Data Cleansing

Pembersihan data (*data cleansing*) merupakan bagian dari proses pra-pemrosesan data. Proses ini mencakup berbagai aktivitas, salah satunya adalah pembersihan data. Dalam proses ini, bagian data yang tidak sempurna, tidak benar, tidak lengkap, tidak akurat, atau tidak relevan diidentifikasi. Jenis data yang “kotor” ini dapat diganti, dimodifikasi, atau dihapus menggunakan teknik-teknik tertentu. Pembersihan data juga dikenal dengan istilah data cleansing (Joshi & Patel, 2020). Tahapan-tahapan

dalam pra-pemrosesan data yang berfokus pada peningkatan kualitas dataset dengan mengidentifikasi dan mengatasi berbagai masalah, seperti *missing values*, *noise*, dan *outlier* (Saraswat & Raj, 2022).

2.1.2.2 Normalisasi Data

Normalisasi data (*Data Normalization*) adalah teknik transformasi data yang bertujuan untuk menskalakan nilai dataset ke rentang tertentu, biasanya antara 0 dan 1. Proses ini sangat penting ketika berhadapan dengan kumpulan data yang memiliki skala yang bervariasi, karena memastikan bahwa setiap fitur memiliki bobot yang sama dalam analisis (Dhawas, dkk., 2024).

1. **MinMax** : Metode Min-Max mengubah nilai data ke dalam rentang tertentu, biasanya antara 0 dan 1. Rumusnya dituliskan sebagai berikut:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2.1)$$

Keterangan:

- x : nilai asli
- $x_{\min}$: nilai minimum dari atribut
- $x_{\max}$: nilai maksimum dari atribut
- x' : nilai hasil normalisasi

2. **z-Score** Metode ini mentransformasikan data agar memiliki rata-rata nol dan standar deviasi satu. Cocok digunakan saat data memiliki distribusi normal. Rumusnya adalah:

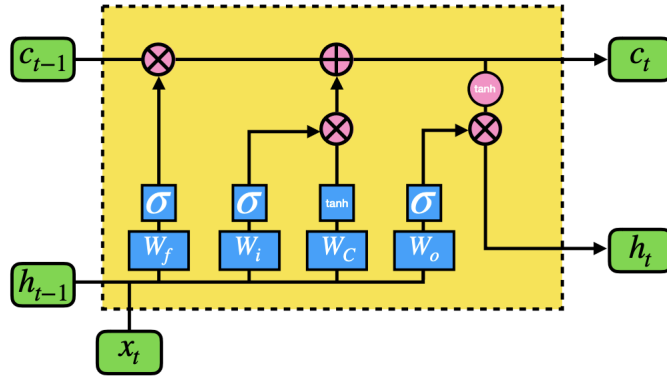
$$z = \frac{x - \mu}{\sigma} \quad (2.2)$$

Keterangan:

- x : nilai asli
- μ : rata-rata dari data
- σ : simpangan baku (standar deviasi)
- z : nilai hasil normalisasi

2.1.3 Long Short-Term Memory (LSTM)

LSTM diperkenalkan pada tahun 1997 oleh Sepp Hochreiter dan Jurgen Schmidhuber sebagai alternatif dari RNN (Recurrent Neural Network) untuk mengatasi masalah hilangnya gradien (*gradient vanishing*), meledaknya gradien (*gradient exploding*), dan ketergantungan jangka panjang (Oguz & Ertugrul, 2023). Pada arsitektur ini, LSTM memiliki kemampuan penyimpanan memori dalam jangka



Gambar 2.1. Arsitektur LSTM
Chen, dkk. (2020b)

waktu relatif lama karena penambahan *memory cell* sehingga dapat menjadi solusi atas terjadinya gradien yang hilang pada RNN selama melakukan proses data sekuensial yang panjang (Pontoh, dkk., 2022). Berikut arsitektur LSTM seperti pada gambar 2.1.

Konsep utama di balik model LSTM adalah mengatur keadaan sel (cell state) dengan menggunakan tiga gerbang, yaitu: *Input gate*, *Forget gate*, dan *Output gate*. *Forget gate* (f_t) menentukan apakah informasi dari keadaan sebelumnya (c_{t-1}) akan dilupakan atau dipertahankan, dengan memperhatikan nilai dari input saat ini (x_t) dan keadaan tersembunyi sebelumnya (h_{t-1}). Nilai keluarannya (f_t) berada di antara 0 dan 1, yang mengindikasikan seberapa besar informasi akan dilupakan (mendekati 0) atau dipertahankan (mendekati 1). *Output gate* (o_t) mengontrol aliran informasi dari keadaan sel saat ini ke keadaan tersembunyi (h_t), dan nilainya berada di antara 0 dan 1. Rincian matematis dari gerbang-gerbang ini diberikan dalam Persamaan (2.1)–(2.5), di mana $x_t \in \mathbb{R}^n$ adalah vektor input, $W \in \mathbb{R}^{v \times n}$ adalah matriks bobot, dan $b \in \mathbb{R}^v$ adalah vektor bias. Superskrip n dan v masing-masing menunjukkan dimensi vektor input dan jumlah kata dalam dataset atau kosakata. Pada setiap waktu t , input ke LSTM terdiri dari vektor input saat ini (x_t), keadaan tersembunyi sebelumnya (h_{t-1}), dan keadaan sel sebelumnya (c_{t-1}), sedangkan outputnya adalah keadaan tersembunyi saat ini (h_t) dan keadaan sel saat ini (c_t). Simbol $\odot$ menyatakan operasi perkalian elemen demi elemen antar vektor (Hameed & Garcia-Zapirain, 2020).

$$f_t = \sigma(W_f x_t + W_{fh} h_{t-1} + b_f) \quad (2.3)$$

$$i_t = \sigma(W_i x_t + W_{ih} h_{t-1} + b_i) \quad (2.4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c x_t + W_{ch} h_{t-1} + b_c) \quad (2.5)$$

$$o_t = \sigma(W_o x_t + W_{oh} h_{t-1} + b_o) \quad (2.6)$$

$$h_t = o_t \odot \tanh(c_t) \quad (2.7)$$

b_n adalah bias yang terkait dengan W_n , $v_t = [h_{t-1} \ x_t]$ merujuk pada *concatenation* antara h_{t-1} dan x_t , dan simbol $*$ serta $+$ masing-masing menyatakan perkalian dan penjumlahan elemen-per-

elemen (*element-wise*) (Chen, dkk., 2020b).

2.1.4 Komputasi Kuantum

Komputasi Kuantum merupakan bagian dari ilmu komputer yang menggunakan prinsip-prinsip teori kuantum seperti superposisi, *entanglement*, *tunneling*, dan lain-lain untuk memproses suatu informasi. Sistem komputasi ini mampu menyelesaikan permasalahan kompleks yang tidak dapat ditangani secara efisien oleh sistem komputasi klasik (Gyongyosi & Imre, 2019). Penyelesaian setiap permasalahan atau tugas-tugas pemrosesan informasi diselesaikan menggunakan sistem mekanika kuantum. Mekanika kuantum sendiri adalah kerangka matematika yang menjelaskan proses fisik pada tingkat mikroskopis. Saat seorang pengamat mengamati sifat-sifat makroskopis suatu sistem, mekanika klasik biasanya sudah mencukupi. Namun, sistem-sistem makroskopis tersebut, jika diamati pada tingkat mikroskopis, tetap berperilaku sesuai dengan hukum-hukum mekanika kuantum (Pattanayak, 2021).

Superposisi merupakan sifat dasar dalam teori kuantum yang memungkinkan sistem kuantum berada secara simultan dalam seluruh kemungkinan keadaan qubit. Sebagai contoh, sebuah qubit tunggal dapat berada dalam keadaan $|0\rangle$ dan $|1\rangle$ secara bersamaan, yang dapat dituliskan sebagai:

$$|\psi\rangle = a|0\rangle + b|1\rangle \quad (2.8)$$

Superposisi dalam mekanika kuantum memungkinkan suatu sistem kuantum untuk berada dalam beberapa keadaan sekaligus. Misalnya, jika suatu sistem kuantum terdiri dari n qubit, maka sistem tersebut memiliki 2^n kemungkinan keadaan komputasi yang dapat disimpan secara bersamaan. Sebaliknya, sistem klasik hanya dapat menyimpan satu nilai dalam satu waktu. Keuntungan utama dari superposisi adalah kemampuannya untuk memungkinkan komputasi kuantum melakukan pemrosesan secara paralel, sehingga meningkatkan kekuatan komputasi secara eksponensial. *Entanglement* (keterikatan kuantum) adalah karakteristik fundamental lain dari teori kuantum, yang menghubungkan dua atau lebih partikel kuantum sehingga keadaan dari satu partikel tergantung pada partikel lainnya. Dalam komputasi kuantum, perubahan pada keadaan satu qubit yang terentang akan langsung memengaruhi keadaan qubit pasangannya. Oleh karena itu, memproses satu qubit yang terentang dapat mengungkap informasi dari qubit lain yang terhubung dengannya (Pandey & Pakray, 2023). *Entanglement* dapat meningkatkan kecepatan pemrosesan komputer kuantum secara signifikan. Sifat ini sangat penting dalam berbagai algoritma kuantum, karena memungkinkan peningkatan kecepatan dibandingkan dengan algoritma klasik.

2.1.4.1 Quantum Bit

Pada sistem komputasi klasik, setiap informasi disimpan dalam bentuk bit (0 atau 1) dan diproses menggunakan gerbang logika digital yang terbatas. Sementara pada sistem komputasi kuantum informasi disimpan dalam bentuk quantum bit atau yang lebih umum disebut sebagai qubit, yang dapat berada dalam superposisi kuantum dari dua keadaan kuantum independen, yaitu 0 dan 1. Ketika

nilai qubit diukur, hasilnya hanya bisa 0 atau 1 karena fungsi gelombang sistem kuantum tersebut mengalami kolaps (Emmanoulopoulos & Dimoska, 2022).

Tabel 2.1. Perbandingan antara komputasi klasik dan kuantum

Aspek	Komputasi Klasik	Komputasi Kuantum
Input	Bit biner	Qubit (bit kuantum)
Komputasi	Rangkaian logika digital	Quantum circuit
Output	Solusi langsung	Quantum measurement

Qubit (Quantum Bit) adalah unit dasar pemrosesan informasi dalam komputasi kuantum yang bersifat analog dengan bit dalam komputasi klasik. Keadaan dasar komputasi dari sebuah qubit dituliskan sebagai:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (2.9)$$

Satu qubit dapat berada dalam kombinasi linier dari kedua keadaan dasar tersebut, yaitu:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \quad (2.10)$$

Di mana α dan β adalah bilangan kompleks yang disebut sebagai amplitudo dari keadaan kuantum. Untuk memastikan bahwa representasi qubit tersebut valid, harus memenuhi kondisi normalisasi berikut:

$$|\alpha|^2 + |\beta|^2 = 1 \quad (2.11)$$

2.1.4.2 Quantum Gates

Dalam komputasi kuantum, sangat penting untuk dapat memanipulasi keadaan sistem kuantum. Mirip dengan sistem klasik di mana keadaan bit dikendalikan menggunakan gerbang logika seperti OR, AND, NOT, XOR, dan lain-lain, sistem kuantum menggunakan gerbang kuantum untuk mengubah keadaan qubit (Pattanayak, 2021).

2.1.4.3 Single-Qubit Gates

Gerbang qubit tunggal mengontrol evolusi keadaan qubit tunggal $|a\rangle$. Sesuai dengan hukum mekanika kuantum, keadaan hasil evolusi harus memenuhi syarat normalisasi. Implikasi dari syarat ini adalah bahwa evolusinya harus berupa operasi unitari. Secara konkret, misalkan $U \in \mathbb{C}^{2 \times 2}$ sebagai operator linear, dan keadaan hasil evolusi dinyatakan sebagai:

$$|\hat{a}\rangle := U|a\rangle = \hat{a}_1|0\rangle + \hat{a}_2|1\rangle \in \mathbb{C}^2 \quad (2.12)$$

Jumlah dari koefisiennya harus memenuhi:

$$|\hat{a}_1|^2 + |\hat{a}_2|^2 = \langle \hat{a} | \hat{a} \rangle = \langle a | U^\dagger U | a \rangle = 1 \quad (2.13)$$

Jika dan hanya jika U adalah unitary, yaitu $U^\dagger U = U U^\dagger = I_2$. Simbol † menunjukkan operasi konjugat-transpos. Dalam representasi operator densitas, evolusi dari $|a\rangle$ menghasilkan:

$$\hat{\rho} = U \rho U^\dagger \quad (2.14)$$

di mana $\hat{\rho} = |\hat{a}\rangle\langle\hat{a}|$ dan $\rho = |a\rangle\langle a|$.

Beberapa gerbang qubit tunggal yang umum termasuk Pauli-X, Pauli-Y, Pauli-Z, Hadamard, serta gerbang rotasi terhadap sumbu X, Y, dan Z (R_X , R_Y , R_Z), seperti yang tercatat pada tabel 2.2 Menurut Teorema 4.1 dalam (Nielsen & Chuang, 2011), setiap operasi unitary pada satu qubit dapat diuraikan menjadi rangkaian rotasi sebagai berikut:

$$U = R_Z(\alpha) \cdot R_Y(\beta) \cdot R_Z(\gamma) \quad (2.15)$$

dimana $\alpha, \beta, \gamma \in [0, 2\pi)$ hingga suatu pergeseran fasa global.

Tabel 2.2. Perbandingan berbagai gerbang kuantum, bentuk matriks, dan representasi sirkuit

Quantum Gate	Matrix Form	Circuit Representation
Pauli-X (X)	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	$\text{---} \boxed{X} \text{---}$
Pauli-Y (Y)	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	$\text{---} \boxed{Y} \text{---}$
Pauli-Z (Z)	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$\text{---} \boxed{Z} \text{---}$
Hadamard (H)	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	$\text{---} \boxed{H} \text{---}$
$R_X(\theta)$	$\begin{bmatrix} \cos(\frac{\theta}{2}) & -i \sin(\frac{\theta}{2}) \\ -i \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}$	$\text{---} \boxed{R_x(\theta)} \text{---}$
$R_Y(\theta)$	$\begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}$	$\text{---} \boxed{R_y(\theta)} \text{---}$
$R_Z(\theta)$	$\begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}$	$\text{---} \boxed{R_z(\theta)} \text{---}$

Tabel 2.2 memuat singkatan, bentuk matematis, dan representasi grafik dari satu set gerbang kuantum universal. Simbol i merepresentasikan bilangan imajiner.

2.1.4.4 Multiple-Qubit Gates

Multiple-Qubit Gates atau gerbang ganda qubit merupakan evolusi keadaan kuantum N-qubit sehingga dapat digeneralisasi secara efektif dari kasus satu qubit (Du, dkk., 2025). Tujuan utama dari gerbang-gerbang ini adalah untuk membangun keadaan keterikatan (*entangled states*) yang kompleks dan memanipulasi keadaan-keadaan tersebut guna menjalankan algoritma kuantum (Zhou, dkk., 2024). Berikut gerbang-gerbang yang menjalankan fungsi dengan input dari beberapa qubit.

1. Gerbang *Controlled-NOT*:

Gerbang *Controlled-NOT* (CNOT) adalah gerbang dua qubit yang umum digunakan dan menerapkan prinsip *entanglement* (Wang, 2012). Gerbang ini bekerja pada satu qubit kontrol dan satu qubit target. Jika qubit kontrol bernilai 1, maka qubit target akan diputar sebesar π radian, sehingga nilainya terbalik (Wang, 2012). Namun, jika qubit kontrol bernilai 0, maka qubit target tetap tidak berubah.

2. Gerbang *Controlled-SWAP* (CSWAP):

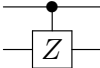
Gerbang *Controlled-SWAP* bekerja pada dua *qubit target* dan satu *qubit kontrol*. Jika qubit kontrol disetel ke 1, maka nilai dari dua qubit target akan saling ditukar (*swap*) (Wang, 2012). Jika qubit kontrol berada dalam keadaan awal 0, maka qubit target tidak mengalami perubahan.

3. Gerbang Toffoli:

Gerbang Toffoli adalah gerbang tiga qubit yang melakukan operasi *NOT* pada qubit target ketika kedua qubit kontrol berada dalam keadaan $|1\rangle$ (Wang, 2012). Jika tidak, qubit target tetap tidak berubah. Gerbang Toffoli dapat dianggap sebagai versi umum dari gerbang CNOT.

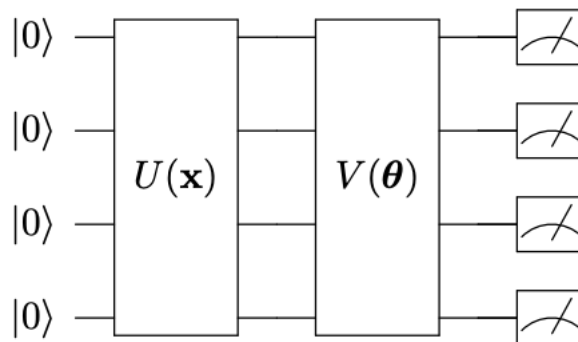
Berikut representasi dari beberapa gerbang-gerbang ganda lihat pada tabel berikut

Tabel 2.3. Tabel gerbang kuantum dua qubit: representasi matriks dan diagram kuantum.

Gate	Matrix Representation	Quantum Circuit
SWAP	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	
Controlled-NOT (CNOT)	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$	
Controlled-Z (CZ)	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$	

2.1.5 Variational Quantum Circuit

VQC (*Variational Quantum Circuits*) adalah jenis sirkuit kuantum yang memiliki parameter yang dapat disesuaikan dan dioptimalkan secara iteratif (Chen, dkk., 2020a). VQC termasuk ke dalam sirkuit kuantum yang memanfaatkan berbagai jenis gerbang logika kuantum untuk melakukan beragam komputasi kuantum (Zhou, dkk., 2024). Tujuan utama penggunaan VQC adalah untuk menemukan nilai optimal dari variabel-variabel yang meminimalkan fungsi biaya (cost function). Fungsi biaya ini dapat merepresentasikan berbagai macam masalah optimasi, seperti mencari keadaan energi terendah dari suatu sistem molekuler atau mengoptimalkan parameter dalam model pembelajaran mesin (Pandey & Pakray, 2023). Berikut gambar 2.2 VQC arsitektur secara general.



Gambar 2.2. Variational Quantum Circuit
Chen, dkk. (2020b)

Pada gambar 2.2 ditunjukkan arsitektur umum dari VQC. Pada arsitektur ini, terdapat blok $U(x)$ yang berfungsi untuk *state preparation*, yaitu mengkodekan data klasik x ke dalam keadaan kuantum dari sirkuit, dan bagian ini tidak dioptimalkan. Sementara itu, blok $V(\theta)$ mewakili bagian variatif dengan parameter yang dapat dipelajari (θ), yang akan dioptimalkan melalui metode gradien. Terakhir, dilakukan *measurement* atau pengukuran terhadap sebagian (atau seluruh) qubit untuk memperoleh *bit string* klasik, seperti 0100 (Chen, dkk., 2020b).

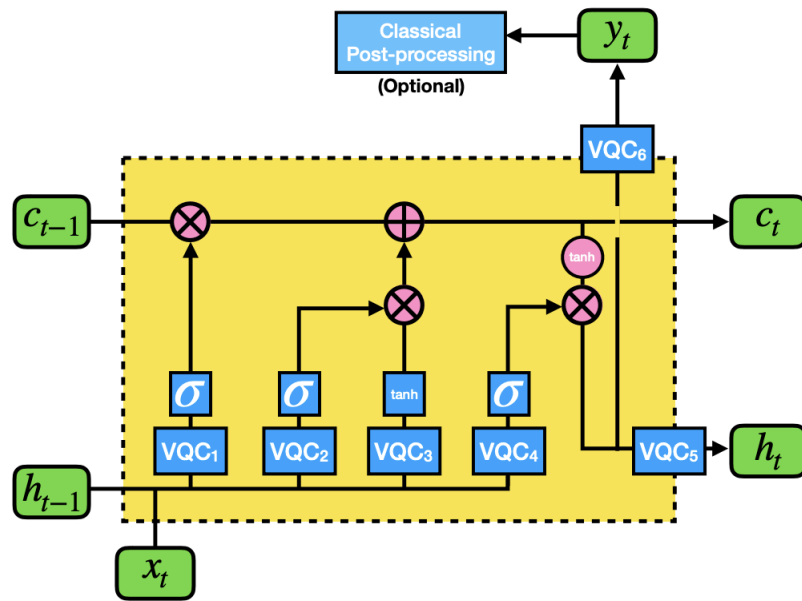
2.1.6 Quantum Long Short-Term Memory

Model LSTM Kuantum (QLSTM) merupakan perpaduan mutakhir antara prinsip-prinsip komputasi kuantum dan jaringan Long Short-Term Memory (LSTM). Tujuan utama dari QLSTM adalah memanfaatkan sifat-sifat khas mekanika kuantum, seperti superposisi, keterikatan (entanglement), dan interferensi kuantum, untuk mengatasi keterbatasan yang melekat pada LSTM klasik (Zhou, dkk., 2024).

Model QLSTM terdiri dari beberapa komponen yang saling terhubung dan secara kolektif memproses data berurutan dengan cara yang sangat efisien dan efektif. Proses dimulai dengan *Quantum Embedding Layer*, yang mengkodekan (*encoding*) data input klasik ke dalam keadaan kuantum. Data yang telah dikodekan ini kemudian diteruskan ke *Quantum Memory Cell*, di mana gerbang kuantum melakukan pembaruan keadaan dan mempertahankan informasi. Antarmuka Klasik-Kuantum

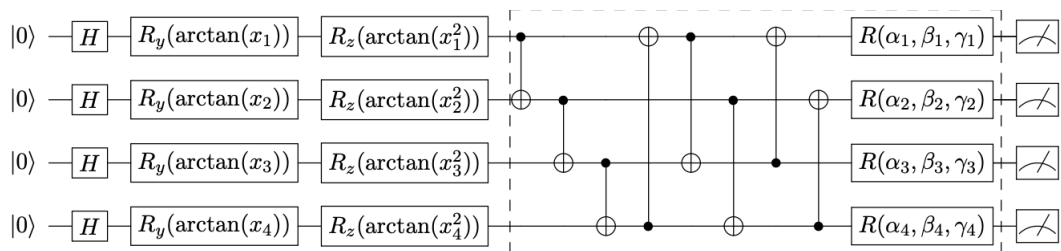
(*Classical-Quantum Interface*) memainkan peran penting dalam memfasilitasi komunikasi antara komponen klasik dan kuantum, yaitu dengan mengkodekan *encoding* data klasik ke dalam keadaan kuantum, serta mengubah kembali keadaan kuantum menjadi data klasik (Zhou, dkk., 2024).

Pada penelitian ini, penulis menggunakan LSTM klasik yang di bawa ke ranah kuantum dengan mengganti jaringan saraf klasik di dalam sel LSTM dengan VQC (Variational Quantum Circuits), yang akan berperan sebagai ekstraktor fitur sekaligus kompresor data. Hal ini seperti yang dilakukan (Chen, dkk., 2020b), pada penelitiannya mereka mengusulkan skematik dari arsitektur QLSTM yang dapat dilihat pada gambar 2.3 berikut.



Gambar 2.3. Arsitektur QLSTM
Chen, dkk. (2020b)

Arsitektur QLSTM yang digunakan seperti pada gambar 2.3 di atas. Setiap kotak *VQC* memiliki bentuk seperti yang dijelaskan pada gambar 2.4. Blok σ dan $\tanh$ merepresentasikan fungsi aktivasi sigmoid dan tangen hiperbolik, masing-masing. x_t adalah *input* pada waktu t , h_t adalah *hidden state* (keadaan tersembunyi), c_t adalah *cell state* (keadaan sel), dan y_t adalah *output*. Simbol $\otimes$ dan $\oplus$ merepresentasikan perkalian dan penjumlahan elemen-per-elemen (*element-wise*), masing-masing.



Gambar 2.4. Variational Quantum Circuit QLSTM Chen, dkk. (2020b)

Gambar 2.4 merupakan arsitektur VQC Generik untuk QLSTM yang diusulkan Chen, dkk. (2020b).

Arsitektur ini terdiri dari tiga lapisan: lapisan penyandian data (menggunakan gerbang H , R_y , dan R_z), lapisan variational (ditandai dengan kotak putus-putus), dan lapisan pengukuran kuantum. Perlu dicatat bahwa jumlah qubit dan jumlah pengukuran dapat disesuaikan dengan permasalahan yang sedang ditangani, dan lapisan variational dapat terdiri dari beberapa kotak putus-putus untuk meningkatkan jumlah parameter, semuanya tergantung pada kapasitas dan kapabilitas mesin kuantum yang digunakan dalam eksperimen.

Untuk membangun unit dasar dari arsitektur QLSTM yang diusulkan, yaitu sebuah *QLSTM cell*, blok-blok *VQC* yang telah dijelaskan sebelumnya disusun secara bertumpuk. Pada gambar 2.3 terdapat enam *VQC* dalam satu sel QLSTM. Untuk VQC_1 hingga VQC_4 , input-nya adalah penggabungan (*concatenation*) v_t dari *hidden state* h_{t-1} pada waktu sebelumnya dan vektor input saat ini x_t . Output-nya berupa empat vektor yang diperoleh dari hasil pengukuran pada akhir masing-masing *VQC*. Nilai hasil pengukuran tersebut yang dirancang sebagai nilai ekspektasi Pauli-Z dari setiap qubit kemudian diteruskan ke fungsi aktivasi nonlinier, yaitu *sigmoid* dan *tanh*.

Formula matematis dari sel QLSTM yang diusulkan oleh Chen, dkk. (2020b) adalah sebagai berikut.

$$f_t = \sigma(VQC_1(v_t)) \quad (2.15)$$

$$i_t = \sigma(VQC_2(v_t)) \quad (2.16)$$

$$\tilde{C}_t = \tanh(VQC_3(v_t)) \quad (2.17)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (2.18)$$

$$o_t = \sigma(VQC_4(v_t)) \quad (2.19)$$

$$h_t = VQC_5(o_t \odot \tanh(C_t)) \quad (2.20)$$

$$y_t = VQC_6(o_t \odot \tanh(C_t)) \quad (2.21)$$

Formula matematis QLSTM di atas dapat dikelompokkan menjadi tiga lapisan berdasarkan fungsinya:

- **Forget Block** [Persamaan (2.15)]: Blok VQC_1 menerima input v_t dan menghasilkan vektor f_t dengan nilai dalam interval $[0, 1]$ melalui fungsi sigmoid. Tujuan dari f_t adalah untuk menentukan apakah elemen-elemen tertentu pada cell state C_{t-1} dari langkah sebelumnya akan ”dilupakan” atau ”dipertahankan”, dengan melakukan operasi elemen-per-elemen pada C_{t-1} (yakni, $f_t \odot C_{t-1}$). Sebagai contoh, nilai 1 (atau 0) menunjukkan bahwa elemen terkait dalam cell state akan sepenuhnya dipertahankan (atau dilupakan). Secara umum, vektor tersebut tidak hanya terdiri dari 0 atau 1, tetapi juga nilai di antaranya, yang berarti sebagian informasi pada cell state akan tetap dipertahankan. Hal ini membuat QLSTM cocok untuk mempelajari atau memodelkan dependensi temporal.
- **Input & Update Block** [Persamaan (2.16)–(2.18)]: Tujuan dari bagian ini adalah untuk menentukan informasi baru yang akan ditambahkan ke cell state. Terdapat dua blok *VQC* dalam bagian ini. Pertama, VQC_2 memproses v_t , dan hasilnya dilewatkan melalui fungsi sigmoid

untuk menentukan nilai-nilai mana yang akan ditambahkan ke cell state. Sementara itu, VQC_3 juga memproses input yang sama dan hasilnya dilewatkan melalui fungsi tanh untuk menghasilkan kandidat cell state baru $\tilde{C}_t$. Akhirnya, hasil dari VQC_2 dikalikan secara elemen-per-elemen dengan $\tilde{C}_t$, dan vektor hasilnya digunakan untuk memperbarui cell state.

- **Output Block** [Persamaan (2.19)–(2.21)]: Setelah cell state diperbarui, sel QLSTM siap untuk menentukan output-nya. Pertama, VQC_4 memproses v_t dan hasilnya dilewatkan melalui fungsi sigmoid untuk menentukan bagian mana dari cell state C_t yang relevan untuk output. Cell state itu sendiri dilewatkan melalui fungsi tanh dan kemudian dikalikan secara elemen-per-elemen dengan hasil dari VQC_4 . Nilai ini kemudian dapat diproses lebih lanjut dengan VQC_5 untuk mendapatkan hidden state h_t , atau dengan VQC_6 untuk menghasilkan output y_t .

2.1.7 Evaluasi Model

Evaluasi Model dalam hal ini metrik akurasi dalam *forecasting time series* mengukur perbedaan antara nilai prediksi dan nilai aktual selama suatu periode, serta memberikan ukuran kuantitatif terhadap kinerja model dalam menangkap pola dan tren temporal (Kolambe, 2024). Pendekatan konvensional untuk menilai keunggulan hasil prediksi dari model yang diusulkan adalah dengan memilih model berdasarkan jumlah *error* prediksi yang paling rendah. Namun, memilih fungsi loss tertentu sebagai metrik evaluasi untuk perbandingan hasil prediksi merupakan hal yang menantang (Iqbal, dkk., 2024).

2.1.7.1 Koefisien Determinasi R^2 Score

Salah satu cara umum untuk merangkum seberapa baik suatu model prediktif menjelaskan variasi dalam data adalah melalui koefisien determinasi, atau R^2 . Nilai ini mencerminkan seberapa besar proporsi variabilitas dalam nilai aktual (y) yang dapat dijelaskan oleh hasil prediksi model ($\hat{y}$). Nilai R^2 dapat dihitung sebagai kuadrat dari korelasi antara nilai observasi aktual dan nilai prediksi (Hyndman & Athanapoulos, 2018). Secara matematis, R^2 juga dapat dihitung menggunakan persamaan 2.16

$$R^2 = 1 - \frac{\sum_{t=1}^N e_t^2}{\sum_{t=1}^N (a_t - \bar{a})^2} \quad (2.16)$$

Di mana:

- R^2 adalah koefisien determinasi, yang mencerminkan proporsi variasi dalam data yang dapat dijelaskan oleh model.
- N menyatakan jumlah total observasi atau titik data yang digunakan dalam evaluasi model.
- e_t adalah galat (error) pada titik ke- t , dihitung sebagai selisih antara nilai aktual dan nilai prediksi, yaitu $e_t = a_t - \hat{a}_t$.
- a_t merupakan nilai aktual dari variabel target pada titik ke- t .

- $\hat{a}_t$ adalah nilai prediksi yang dihasilkan oleh model pada titik ke- t .
- $\bar{a}$ adalah rata-rata dari seluruh nilai aktual, dirumuskan sebagai:

$$\bar{a} = \frac{1}{N} \sum_{t=1}^N a_t$$

- Penyebut pada rumus R^2 , yaitu $\sum_{t=1}^N (a_t - \bar{a})^2$, dikenal sebagai *Total Sum of Squares* (TSS) dan merepresentasikan total variasi dalam data aktual.
- Pembilangnya, yaitu $\sum_{t=1}^N e_t^2$, disebut sebagai *Residual Sum of Squares* (RSS), yang mencerminkan jumlah kesalahan prediksi model.

Rumus ini mengukur sejauh mana galat prediksi (*residual sum of squares*) dibandingkan dengan total variasi dalam data (*total sum of squares*). Nilai R^2 berada pada rentang 0 hingga 1, di mana nilai mendekati 1 menunjukkan bahwa model mampu menjelaskan sebagian besar variasi dalam data, sedangkan nilai mendekati 0 menunjukkan bahwa model memiliki kemampuan prediksi yang rendah.

2.1.7.2 Mean Absolute Error

Forecast Error atau galat peramalan berada pada skala yang sama dengan data. Oleh karena itu, ukuran akurasi yang hanya didasarkan pada galat bersifat tergantung skala (*scale-dependent*) dan tidak dapat digunakan untuk membandingkan antar deret waktu yang memiliki satuan berbeda. Saat membandingkan metode peramalan yang diterapkan pada satu deret waktu, atau pada beberapa deret waktu dengan satuan yang sama, MAE (*Mean Absolute Error*) sering digunakan karena mudah dipahami dan dihitung (Hyndman & Athanasopoulos, 2018). Berikut persamaan matematis MAE 2.17

$$MAE = \frac{1}{N} \sum_{t=1}^N |e_t| \quad (2.17)$$

Di mana :

- N adalah jumlah total observasi atau titik data.
- e_t adalah galat (error) pada waktu atau titik ke- t , yang didefinisikan sebagai $e_t = y_t - \hat{y}_t$.
- y_t adalah nilai aktual dari variabel target pada titik ke- t .
- $\hat{y}_t$ adalah nilai prediksi dari model pada titik ke- t .
- $|e_t|$ adalah nilai absolut dari galat pada titik ke- t .

Metode peramalan yang meminimalkan MAE (Mean Absolute Error) akan menghasilkan prediksi yang mengarah pada nilai median. MAE menghasilkan rata-rata dari seluruh kesalahan absolut. Nilai MAE yang lebih kecil menunjukkan performa prediksi yang lebih akurat.

2.1.7.3 Mean Square Error

Mean Squared Error (MSE) adalah ukuran akurasi yang dihitung dengan mengkuadratkan setiap galat (error) individu dalam sebuah himpunan data, kemudian mengambil nilai rata-rata dari jumlah kuadrat tersebut. MSE memberikan bobot yang lebih besar terhadap galat-galat besar dibandingkan galat-galat kecil karena setiap galat dikalikan dengan dirinya sendiri sebelum dijumlahkan (Wheelwright, dkk., 1998).

$$MSE = \frac{1}{N} \sum_{t=1}^N e_t^2 \quad (2.18)$$

Di mana :

- MSE: *Mean Squared Error*, yaitu rata-rata dari kuadrat galat (kesalahan) prediksi.
- N : jumlah total observasi atau titik data.
- e_t : galat (kesalahan) prediksi pada waktu ke- t , yang didefinisikan sebagai:

$$e_t = a_t - \hat{a}_t$$

- a_t : nilai aktual pada waktu ke- t .
- $\hat{a}_t$: nilai prediksi (forecast) pada waktu ke- t .

2.1.7.4 Mean Absolute Percentage Error

Mean Absolute Percentage Error merupakan rata-rata dari jumlah seluruh persentase galat (error) dalam suatu himpunan data, tanpa memperhatikan tanda positif atau negatifnya. Artinya, nilai absolut dari setiap persentase galat dijumlahkan, lalu dirata-ratakan. MAPE merupakan salah satu ukuran akurasi yang umum digunakan dalam metode kuantitatif untuk peramalan (Wheelwright, dkk., 1998).

$$MAPE = \frac{1}{N} \sum_{t=1}^N \left| \frac{e_t}{p_t} \right| \times 100\% \quad (2.19)$$

Di mana:

- MAPE : Mean Absolute Percentage Error, yaitu rata-rata kesalahan absolut dalam bentuk persentase terhadap nilai aktual.
- N : Jumlah total observasi dalam data.
- t : Indeks waktu atau pengamatan, dengan $t = 1, 2, \dots, N$.
- e_t : Galat (error) pada waktu t , dihitung sebagai $e_t = a_t - \hat{a}_t$.
- a_t : Nilai aktual pada waktu t .

- $\hat{a}_t$: Nilai prediksi pada waktu t .
- p_t : Nilai aktual pada waktu t (bisa ditulis juga sebagai a_t).
- $|\cdot|$: Notasi nilai absolut, digunakan untuk menghilangkan tanda negatif/positif dari galat.
- $\times 100\%$: Mengubah hasil menjadi persentase.

Berikut beberapa evaluasi metrik yang digunakan seperti di atas yang menunjukkan bahwa nilai R^2 dapat berkisar dari 0 hingga 1, dengan nilai yang lebih tinggi menunjukkan kinerja prediksi model yang lebih baik. R^2 , yang juga disebut rasio pengembalian yang disesuaikan dengan risiko dalam bidang keuangan, membantu investor dalam menilai investasi yang ada maupun yang potensial. Selain itu, *Mean Absolute Error* (MAE) dan *Mean Squared Error* (MSE) mengukur perbedaan antara nilai aktual dan nilai prediksi. Oleh karena itu, memilih model dengan nilai MSE dan MAE terendah dapat menghasilkan hasil prediksi yang lebih akurat. MSE menghitung kuadrat dari kesalahan sebelum dirata-ratakan, sehingga memberikan bobot yang lebih besar terhadap kesalahan yang besar. MSE sangat cocok digunakan dalam prediksi nilai tukar, di mana kesalahan besar sangat tidak diinginkan. Sebaliknya, MAE tidak terlalu sensitif terhadap pencilan jika dibandingkan dengan MSE, namun tetap merupakan metrik evaluasi yang berguna untuk data deret waktu (*time series*). *Mean Absolute Percentage Error* (MAPE) digunakan sebagai metrik loss nonlinier dengan rentang nilai $[0, +\infty)$. Semakin kecil nilai MAPE, semakin baik kinerja prediksi model (Iqbal, dkk., 2024).

